Retrieval Pipeline Experiments

Deep Research Agent · settling query decomposition + retrieval before claim/verification

2026-06-10

Retrieval Pipeline Experiments (2026-06-10)

1. Why these experiments

Inspecting the Phase-2 walking-skeleton run exposed two upstream defects that poison everything downstream of retrieval:

1. **The decomposer always split.** The planner prompt hardcoded "3–6 sub-questions," so even an atomic query was fragmented, with no notion of whether the sub-questions retrieve genuinely distinct evidence.
2. **Retrieval starved the generator.** Exa was called in full-text mode (~60k-char whole pages), then each passage was truncated to its first 600 characters — i.e. the generator wrote claims off a paper's title + author list + first sentence of the abstract.

These sit *upstream* of claim extraction and verification, so there is no point hardening the verifier while the generator is fed the wrong evidence. This set of experiments settles **query decomposition** and **retrieval** (excerpt budget + depth) with data, so subsequent claim-extraction / verification work builds on a defended foundation.

Settled defaults (headline): decomposition **ON** · excerpt budget **maxCharacters = 2,000** · retrieval depth **k = 5** records per sub-question.

2. Setup

Component	Choice
Generator / planner	Mistral Large 3 (Bedrock)
Retrieval	Exa, query-targeted verbatim **highlights** (not full text)

Component	Choice
Independent judge	Llama 4 Maverick (Bedrock) — distinct family from the generator
Test cases	The four brief prompts (below)
Harness	`experiments/decomposition_retrieval.py` (staged, human-in-the-loop)
Results	`experiments/results/stage{1..4}_*/` (per-case JSON + summary + verdict)

Test cases (from the assessment brief), chosen to span the broad↔specific axis:

1. *Straightforward* — real-world risks/benefits of synthetic data for training/fine-tuning LLMs.
2. *Contradictory sources* — is chain-of-thought effective reasoning or mainly output formatting?
3. *Sparse & emerging* — current state of inference-time compute scaling; validated vs. speculative.
4. *Multi-modal output* — map the multi-agent LLM systems landscape; include a visual taxonomy.

Method note — Exa highlights. We call Exa with `contents.highlights` and a per-source `maxCharacters` cap; it returns the verbatim excerpts most relevant to the (sub-)query, ranked. This preserves the invariant that citations are real retrieved text (extractive), not model-paraphrased, while keeping context small.

3. Experiment 1 — Query decomposition

Design (Stage 1): for each test case, compare two retrieval arms at a fixed 2k budget — **A: no decomposition** (one search on the full query) vs **B: decomposition** (plan → one search per sub-question). Measure unique sources, source overlap, and inspect quality.

Results:

Test case	sub-qs	A sources	B sources	only-B	Jaccard	gain (B–A)
tc1 straightforward	4	8	29	23	0.19	+21
tc2 contradictory	5	8	30	25	0.15	+22
tc3 sparse	4	8	27	23	0.13	+19

Test case	sub-qs	A sources	B sources	only-B	Jaccard	gain (B–A)
tc4 multimodal	5	8	36	35	0.02	+28

Findings:

- Decomposition surfaces **3–4× more unique sources** with low overlap, and inspection shows the extra sources are *relevant*, including canonical primaries the broad query misses (e.g. tc2's Arm B retrieved the original CoT paper [2201.11903] under its "evidence CoT improves reasoning" sub-question; Arm A's top hit was a secondary reflection).
- Sub-questions are genuinely distinct, not padding. The **contradiction case (tc2)** is the standout: it split into pro-evidence / con-evidence / methodological-differences / conditions / reconciliation — deliberately retrieving *both poles plus the explanation*, exactly how a contradiction should be researched. (An earlier hypothesis that decomposition would *hurt* tc2 was wrong.)
- **Atomic-query check ("return 1" path):** two single-fact probes — *"maximum context window of Llama 4 Maverick"* and *"year the Transformer paper was published"* — each returned exactly **1 sub-question, unchanged**. The flexible decomposer does not over-split.

Decision: KEEP decomposition. It broadens coverage with distinct, relevant evidence and passes atomic queries through unchanged.

4. Experiment 2 — Excerpt budget (maxCharacters)

Design (Stage 2): with decomposition on, sweep the per-record excerpt budget **1k / 2k / 4k** and compare the same source's excerpt across budgets.

Results: the source *set* is stable across budgets (~25–37/query) — `maxCharacters` changes only how much text per source. Total evidence text scales ~linearly:

Budget	total excerpt chars / query	per synthesis call (k=5)
1k	~30k	~5k chars (~1.3k tok)
2k	~60–77k	~10k chars (~2.5k tok)
4k	~120–158k	~20k chars (~5k tok)

Findings: 1k already returns substantive on-topic spans (not front-matter); 2k adds useful framing (definitions, related work, comparative findings) with little redundancy; 4k roughly doubles text with diminishing returns and begins overlapping what 2k captured.

Decision: maxCharacters = 2,000 — smallest budget that reliably captures the substantive span (results + framing) that 1k occasionally clips, without 4k's redundancy/bloat.

5. Experiment 3 — Retrieval depth (k)

A measurement obstacle, surfaced and handled. Exa returns a *ranked* list but **no usable relevance score**: default/auto search omits `score`; `type:neural`'s "score" is exactly normalized rank (`1.0, 0.889, …, 0`), carrying no magnitude. So "score fall-off" is impossible — we judge relevance/coverage with the independent Llama 4 Maverick judge.

Stage 3 — per-record relevance by rank (0/1/2): essentially flat and high through rank 10; **100% of records at least partially relevant at every k**; mean relevance drifts only ~3% from k=5 (1.86) to k=10 (1.81). No quality cliff.

Stage 4 — cumulative coverage (the deciding measure). Per-record relevance answers "is each record on-topic" but not the real question: does the *combined set* of k records answer the sub-question more *completely*? We judge how completely the COMBINED top-k evidence answers each sub-question (nested sets, so completeness only rises — the signal is the *lift*).

Test case	k=5	k=8	k=10	lift 5→8	lift 8→10
tc1 straightforward	80.0	80.0	80.0	+0.0	+0.0
tc2 contradictory	80.0	80.0	80.0	+0.0	+0.0
tc3 sparse	82.5	82.5	82.5	+0.0	+0.0
tc4 multimodal	86.0	84.0	84.0	–2.0	+0.0
average	**82.1**	**81.6**	**81.6**	**–0.5**	**+0.0**

Findings: the combined top-5 answers each sub-question **as completely as top-10** — zero marginal coverage from records 6–10. The "missing aspect" is **identical at k=5 and k=10** (e.g. "quantitative analysis", "meta-analysis", "empirical comparison"): records 6–10 do not close the gap; they are pure redundancy.

Decision: k = 5. Both metrics agree records beyond 5 add cost (synthesis + verification load) with no coverage gain. ~5 × 4–5 sub-questions ≈ 20–25 relevant sources/query — ample.

Secondary insight: the coverage ceiling (~80–85, not 100) is **not a k problem** — the residual gap is a different *type* of evidence (meta-analytic synthesis / quantitative comparison) that more of the same ranked list will never supply. Closing it is a job for query *reformulation* in the adaptive loop, not a larger k.

6. Settled defaults

Parameter	Value	Settled by
Query decomposition	ON (flexible 1…N; split only for distinct evidence)	Experiment 1
Excerpt budget (maxCharacters)	2,000	Experiment 2
Retrieval depth (k)	5 per sub-question	Experiment 3

7. What this sets up next (deferred, by design)

These are *static* defaults — a sensible floor. The **real adaptivity is feedback-driven** and lives in the Phase-3 verification loop, not in upfront guesses:

- The verifier emits both a **label** (supported / contested / insufficient) **and a recommendation** of what's needed.
- The loop acts on that recommendation: **escalate k** (add records), **widen maxCharacters** (more per record), or **reformulate** the sub-question (when the gap is a missing *type* of evidence, as Experiment 3 found) — until evidence stabilizes or a budget/ no-progress termination fires.

This is why the static defaults are deliberately lean: start small, grow on measured demand. Next work: claim extraction + the verifier (A: LLM-judge vs B: fine-tuned cross-encoder) and the adaptive loop.

8. Reproducibility

```
# decomposition A/B at 2k (Experiment 1)
python -m experiments.decomposition_retrieval
# maxCharacters sweep 1k/2k/4k (Experiment 2)
python -m experiments.decomposition_retrieval charsweep
# per-record relevance by rank (Experiment 3, Stage 3)
python -m experiments.decomposition_retrieval ksweep
# cumulative coverage by k (Experiment 3, Stage 4)
python -m experiments.decomposition_retrieval kcoverage
```

Raw outputs + per-stage verdicts: `experiments/results/stage{1..4}_*/`. Harness: `experiments/decomposition_retrieval.py`. Generator/judge via Bedrock; retrieval via Exa.